

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK 7

RECALL LAST WEEKS

Software systems rarely operate in isolation. They talk to databases, web browsers, mobile apps, third-party services, and each other. For that communication to work, both sides must agree on how data is structured, encoded, and decoded. This agreement is what we call a data interchange format.

What Is a Data Interchange Format

- A data interchange format is a set of rules for representing structured data as a sequence of bytes so that it can be stored, transmitted, and reconstructed by another system. The format defines how objects, arrays, strings, numbers, and other data types are written, and how a parser should read them back.
- Think of it as a shared language. If your Python backend speaks JSON and your JavaScript frontend speaks JSON, they can exchange data without confusion. If one speaks JSON and the other speaks XML, you need a translator. Choosing the right format is therefore a design decision that affects performance, readability, and maintainability.

What Is JSON? Explained in 60 Seconds#QuikZen #JSON #API

What Is XML? Explained in 60 Seconds#QuikZen #XML #Programming

Serialization

Serialization is the process of converting an in-memory data structure into a format that can be stored or transmitted. Your Python dictionary, your Java object, your C# class, these live in memory as complex structures with pointers and references. They cannot be written to a file or sent over a network in that form. Serialization flattens them into a sequence of bytes or characters that can be saved or sent.

Think of it like packing furniture for shipping. Your furniture (the data structure) is bulky and awkward. Serialization is the process of disassembling it, wrapping it, and putting it in a box so it can be transported. The box is the serialized form. It can be stored in a warehouse or shipped on a truck.

Deserialization

Deserialization is the reverse. It takes a serialized format, a string or a sequence of bytes, and reconstructs the in-memory data structure from it. The receiver of the data reads the bytes, parses them according to the format rules, and builds the original object back.

Continuing the furniture analogy, deserialization is unpacking the box and reassembling the furniture. The box arrives at its destination, and you open it, take out the pieces, and put them back together into a usable form.

Serialization and deserialization are always a pair. Whatever you serialize must be deserializable by the other side. This is why the format must be agreed upon in advance. If you serialize with JSON and the receiver expects XML, deserialization will fail.

JSON: JavaScript Object Notation

- JSON is a lightweight, text-based format that represents data as key-value pairs and ordered lists. It was derived from JavaScript syntax but is now language-independent and supported by virtually every programming language.
- It is the default format for most APIs, web applications, and configuration files in modern software.

JSON Syntax Rules

RULE	EXAMPLE
Objects use curly braces with key-value pairs	{"name": "Ali", "age": 25}
Keys must be double-quoted strings	"name" is valid, name is not
Arrays use square brackets	["apple", "banana"]
Strings use double quotes	"hello"
Numbers have no quotes	42, 3.14
Booleans are lowercase	true, false
Null is lowercase	null
No comments allowed	// comment is invalid

example

```
{
  "student": {
    "name": “Sara Khan",
    "age": 22,
    "courses": ["SCD", "Data Structures", "Databases"],
    "active": true,
    "gpa": 3.75
  }
}
```

To be practiced in lab

```
import json
# Python dictionary
student = {
    "name": "Ali Khan",
    "age": 22,
    "courses": ["SCD", "Data Structures", "Databases"],
    "active": True,
    "gpa": 3.75
}
# Serialize: Python object to JSON string
json_string = json.dumps(student, indent=2)
print(json_string)
# Deserialize: JSON string to Python object
parsed = json.loads(json_string)
print(parsed["name"])       # Ali Khan
print(parsed["courses"][0]) # SCD
```

Common JSON Operations

OPERATION	PYTHON CODE
Write to file	json.dump(data, open("file.json", "w"))
Read from file	data = json.load(open("file.json"))
Pretty print	json.dumps(data, indent=2)
Sort keys	json.dumps(data, sort_keys=True)
Handle non-ASCII	json.dumps(data, ensure_ascii=False)

JSON offers the best compromise between interoperability and ease of use for most web and API scenarios. It is not the smallest or fastest format, but it is almost always good enough

STRENGTHS	**WEAKNESSES**
Simple syntax, easy to learn	No support for comments
Native support in JavaScript	No schema validation built in
Compact compared to XML	Numbers are limited to floating point
Fast parsing in most languages	No support for dates or binary data
Universal language support	Keys must be strings

XML: eXtensible Markup Language

XML is a markup language that uses tags to describe data. It was designed to be both human-readable and machine-readable, and it supports complex structures such as nested elements, attributes, namespaces, and schema validation. XML was the dominant format for web services in the early 2000s and remains widely used in enterprise systems, configuration files, and document formats such as Microsoft Office files.

XML Syntax Rules

RULE	EXAMPLE
Elements have opening and closing tags	<name>Ali</name>
Elements can have attributes	<student id="1">
Elements can be nested	<student><name>Ali</name></student>
Every document has one root element	<students>...</students>
Tags are case-sensitive	<Name> and <name> are different
Comments use <!-- -->	<!-- this is a comment -->
Namespaces use xmlns	xmlns:ns="http://example.com"

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
   <student id="1">
      <name>Sara Khan</name>
      <age>22</age>
      <courses>
         <course>SCD</course>
         <course>Data Structures</course>
      </courses>
      <active>true</active>
      <gpa>3.75</gpa>
   </student>
</students>
```

Python provides xml.etree.ElementTree in the standard library for parsing and creating XML

```
import xml.etree.ElementTree as ET
# Parse XML from a string
xml_string = """
<students>
   <student id="1">
      <name>Ali Khan</name>
      <age>22</age>
      <gpa>3.75</gpa>
   </student>
</students>
"""
root = ET.fromstring(xml_string)
# Access elements
for student in root.findall("student"):
   name = student.find("name").text
   age = student.find("age").text
   gpa = student.find("gpa").text
   student_id = student.get("id")
   print(f"ID: {student_id}, Name: {name}, Age: {age}, GPA: {gpa}")
```

Creating xml in python

```
import xml.etree.ElementTree as ET

root = ET.Element("students")
student = ET.SubElement(root, "student", id="1")

name = ET.SubElement(student, "name")
name.text = "Ali Khan"

age = ET.SubElement(student, "age")
age.text = "22"

tree = ET.ElementTree(root)
ET.indent(tree, space="  ")
tree.write("students.xml", encoding="utf-8", xml_declaration=True)
```

Strengths	Weaknesses
Supports schema validation (XSD)	Verbose and bulky
Supports namespaces	Slower to parse than JSON
Supports comments	Complex syntax
Good for document-centric data	Overkill for simple data exchange
Mature tooling and standards	Not natively supported in JavaScript

JSON is generally preferred for browser-side and API communication because its files are smaller and parse faster. XML is preferred in server-side enterprise systems where validation, namespaces, and document structure matter more than size.

Feature	JSON	XML
Data model	Key-value pairs and arrays	Tags and attributes
Readability	Easy	Moderate to difficult
File size	Smaller	Larger due to tags
Parsing speed	Fast	Slower
Schema support	JSON Schema (optional)	XSD (standardized)
Comments	Not supported	Supported
Namespaces	Not supported	Supported
Native JavaScript support	Yes	No
Best for	Web APIs, config files	Enterprise systems, documents

Modern Alternatives

While JSON and XML dominate, several other formats have emerged for specific needs. The choice depends on whether you prioritize size, speed, readability, or schema enforcement.

YAML

- YAML is a human-readable format often used for configuration files. It uses indentation instead of braces and brackets, which makes it more readable than JSON for complex configurations. YAML is popular in DevOps tools such as Docker Compose, Kubernetes, and CI/CD pipelines.

Yaml syntax example

```
student:
  name: Sara Khan
  age: 22
  courses:
    - SCD
    - Data Structures
  active: true
  gpa: 3.75
```

- YAML is more functional and readable than JSON, but JSON is much faster for serialization and deserialization. Use YAML for human-written configuration, and JSON for machine-to-machine exchange.

Protocol Buffers (Protobuf)

Protocol Buffers is a binary format developed by Google. It uses a schema file (.proto) to define the structure of messages, and a compiler generates code in the target language for serialization and deserialization. Because the schema is separate from the data, payloads are much smaller than JSON or XML.

Research shows that Protocol Buffers excels in compactness and speed within Java and Python environments. It is ideal for high-performance microservices, gRPC APIs, and situations where bandwidth and latency matter.

protobuf

```
// student.proto
message Student {
    string name = 1;
    int32 age = 2;
    repeated string courses = 3;
    bool active = 4;
    float gpa = 5;
}
```

Apache Avro

Avro is a binary format that stores the schema alongside the data, which makes it self-describing. It supports schema evolution, meaning you can add or remove fields without breaking compatibility. Avro is widely used in big data systems such as Apache Hadoop and Apache Kafka.

MessagePack

MessagePack is a binary format that aims to be as simple as JSON but more compact. It is useful when you want JSON-like simplicity with better performance. However, industry demand for MessagePack remains lower than for Protobuf and Avro.

Cap'n Proto

Cap'n Proto is a binary format designed for extreme speed. Its documentation claims it is "infinity times faster" than Protocol Buffers in some benchmarks because it supports zero-copy deserialization, meaning data can be read directly from the wire without parsing. It is used in performance-critical systems.

ULMEN

ULMEN is a newer serialization format built in Rust, designed for AI agent communication and LLM context. It claims to be 8x faster than JSON while producing payloads 55-97% smaller. This is an emerging format, and its long-term adoption is not yet established

EXPLORE IF IT INTERESTS YOU

SCENARIO	RECOMMENDED FORMAT	WHY
Public REST API	JSON	Universal support, easy to debug
Internal microservices	Protobuf or gRPC	Small payloads, fast serialization
Configuration files	YAML	Human-readable, supports comments
Enterprise integration	XML	Schema validation, namespaces
Big data pipelines	Avro or Parquet	Schema evolution, columnar storage
Real-time systems	Cap'n Proto or Protobuf	Extreme speed, low latency
Browser-to-server	JSON	Native JavaScript support
Legacy system integration	XML	Existing tooling and standards

Some Keypoints

- Data interchange formats are a foundational concern in software construction.
- The format you choose affects how fast your system runs, how easy it is to debug, and how well it integrates with other systems.
- JSON and XML remain the workhorses, but binary formats such as Protobuf and Avro are increasingly important in performance-critical and data-intensive applications.
- Understanding the tradeoffs allows you to make informed decisions rather than defaulting to whatever is familiar.

LAB

Write a Python script that reads a JSON file containing a list of students, filters those with a GPA above 3.5, and writes the result to a new JSON file. Write a second script that performs the same task using XML instead of JSON. Compare the amount of code required in each case. Then write a third script that serializes the same data using Protocol Buffers and compare the file size of the three outputs. This exercise will give you hands-on intuition for the tradeoffs discussed in this lecture.

For queries

Office : B05, RSB

Counselling Hours :

Monday	11:00 am -12:00 pm
Tuesday	12:00 pm -1:00 pm
Thursday	10:15 am -11:15 am

mumer.haroon@imsciences.edu.pk

Muhammad Umer Haroon | LinkedIn